

UnlearnX: Accelerated Implementation Blueprint & 1-Month Roadmap

Prepared for: Technical Execution Team **Based on:** UnlearnX Project Concept (Simar Ahluwalia, Amey Ghatol)

1. Executive Summary & Strategy

The **UnlearnX** framework shifts enterprise compliance unlearning from destructive base-model weight retraining to a real-time, parameter-efficient inference routing layer using dynamic Low-Rank Adapters (LoRAs).

To compress the initial exploratory scope into a strict **1-month (4-week) sprint**, we will scope the target models down to lightweight variants (e.g., Llama-3.2-1B-Instruct or Mistral-7B-Instruct-v0.3 using a single A100/H100 or dual RTX 4090 GPUs) and utilize a pre-curated subset of the **TOFU (Task of Fictitious Unlearning)** benchmark for fast iteration.

2. Core Components to Be Built

To make UnlearnX operational within 30个小时/week development cycles over 4 weeks, four primary software components must be engineered:

Component A: Activation Localization Probe (ALP)

- **Purpose:** Forward-passes target forget data through the frozen base model to identify which layers and attention heads store the memorized concepts.
- **Implementation:** PyTorch hook implementation (`register_forward_hook`) on intermediate transformer layers to map key-value activation clusters.

Component B: Optimization Engine (Orthogonal Shift + KL Anchor)

- **Purpose:** Calculates weight updates that remove target data without harming general knowledge.
- **Implementation:** * *Gradient Reversal / Orthogonal Shift*: Projects gradients of the target data onto the orthogonal complement of the model's retained knowledge space.
 - *Retain-Set KL Anchor*: Computes Kullback-Leibler divergence loss on a baseline retain set to prevent catastrophic forgetting.

Component C: UnlearnX API Gateway & Orchestrator

- **Purpose:** Exposes a clean REST API interface for compliance officers.
- **Implementation:** FastAPI application handling POST /unlearn requests, spinning up the optimization pipeline asynchronously, saving the generated LoRA weights (.safetensors), and registering them with the serving layer.

Component D: vLLM Hot-Swap Router Integration

- **Purpose:** Instantly switches or injects the newly generated unlearning LoRA adapter into live production inference without restarting the base model.
- **Implementation:** Integration with vLLM's dynamic LoRA loading/unloading endpoints or programmatic adapter management via vLLM engine API.

3. Realistic 1-Month Execution Roadmap (Sprint Plan)

Week 1: Environment Setup, Base Model & ALP Engine

- **Task 1.1:** Initialize the repository, configure PyTorch, Hugging Face transformers/peft, and install vLLM server environment.
- **Task 1.2:** Download and preprocess the TOFU dataset (subset targeting 20 fictitious author profiles for controlled testing).
- **Task 1.3:** Build and validate the **Activation Localization Probe (ALP)** script to extract activation metrics across model layers.
- **Milestone 1:** Successfully run a probe pass and output layer-sensitivity heatmaps for target forget phrases.

Week 2: Optimization Engine & Loss Functions

- **Task 2.1:** Implement the custom training loop combining Gradient Reversal / Orthogonal projection.
- **Task 2.2:** Integrate the **Retain-Set KL Anchor Loss** function using a small validation split to anchor baseline probabilities.
- **Task 2.3:** Write the PEFT LoRA configuration wrapper ($r=16$, $\text{lo_ra_alpha}=32$) targeting attention projections (q_proj , v_proj).
- **Milestone 2:** Generate a functional Unlearning LoRA adapter locally and verify that direct generation on forget data drops significantly.

Week 3: API Gateway & vLLM Hot-Swapping Integration

- **Task 3.1:** Build the **FastAPI Gateway** handling asynchronous task queues (celery or Python asyncio) for unlearning jobs.
- **Task 3.2:** Connect the pipeline output (.safetensors file) to the vLLM server's dynamic adapter loading module.
- **Task 3.3:** Build automated verification script to confirm zero-downtime hot-swapping.
- **Milestone 3:** End-to-end API test: hitting POST /unlearn triggers optimization, saves adapter, and vLLM immediately serves subsequent requests using the new adapter.

Week 4: Evaluation, KPI Verification & Hardening

- **Task 4.1:** Run **Forget Quality Tests (MIA)**: Test resistance to membership inference attacks and adversarial jailbreaks.
- **Task 4.2:** Run **Retain Utility Tests**: Evaluate model performance degradation against a

baseline MMLU subset (< 1% drop target).

- **Task 4.3:** Benchmark end-to-end latency: ensure request-to-injection finishes within the <300 second threshold for a 1MB corpus.
- **Milestone 4:** Final PoC demonstration, documentation, and performance report.

4. Technical Specifications & API Contract

FastAPI Endpoint Spec

POST /unlearn

Content-Type: application/json

```
{
  "request_id": "req_982341",
  "target_corpora": [
    "Confidential PII or copyrighted text block to be excised..."
  ],
  "model_id": "meta-llama/Llama-3.2-1B-Instruct",
  "retention_weight": 0.5
}
```

Response (200 OK):

```
{
  "status": "success",
  "adapter_id": "unlearn_lora_req_982341",
  "latency_seconds": 184.2,
  "metrics": {
    "mia_score_drop": 0.89,
    "mmlu_degradation_pct": 0.42
  },
  "message": "Unlearning LoRA generated and hot-swapped into vLLM router successfully."
}
```

5. Risk Management & Mitigations for 1-Month Sprint

1. **Risk:** Optimization time exceeds 300 seconds on consumer/mid-tier GPUs.
 - *Mitigation:* Restrict rank ($r=8$) and limit target modules exclusively to attention value projections (v_{proj}).
2. **Risk:** Catastrophic forgetting of unrelated knowledge during orthogonal gradient shifts.
 - *Mitigation:* Increase the KL Divergence penalty weighting coefficient relative to the forget loss gradient.
3. **Risk:** vLLM dynamic adapter loading overhead or version conflicts.
 - *Mitigation:* Fallback to a lightweight Hugging Face pipeline-based router simulation if vLLM dynamic multi-adapter serving encounters version blockers.